Flow Logging Architecture — Design Document

Status: Design only. No code changes made yet. **Date:** 2026-05-21 **Scope:** New flow-based tracing layer. Existing `T3_APP_LOG`, `ActivityLogTab`, and all current logging code are untouched.

0. Database placement and migration strategy

0a. Which database gets the flow tables?

Only `webview_t3_device.db` (local SQLite). Never the MSSQL center DB.

This is consistent with the existing design rule: flow logs are local-only diagnostic data, not operational records that need to be replicated to the center database. Every installation — both standalone and server mode — has a local `webview_t3_device.db`, so every installation always gets the tables.

The four SQL schema files in `migration/sql/` serve different purposes:

File	Used by	Flow tables needed?
`webview_t3_device_schema.sql`	SQLite fresh-install path	**YES**
`webview_t3_device_mssql.sql`	Reference / future MSSQL deploy	No
`webview_t3_device_mysql.sql`	Reference / future MySQL deploy	No
`webview_t3_device_postgres.sql`	Reference / future PostgreSQL deploy	No

0b. How the migration system works (existing infrastructure)

```
TEXT

Startup
  └── initialize_t3_device_database()
        ├── [if no DB exists] create from embedded SQL schema OR copy ResourceFile/webvi
        └── run_t3_device_migrations()
              └── T3DeviceMigrator::up(&conn, None)   ← SeaORM
                    ├── reads seaql_migrations table to find already-applied migrations
                    └── applies only NEW migrations in order, skips already-applied ones
```

SeaORM maintains a `seaql_migrations` table inside the SQLite file itself. Every migration has a name derived from its file name (e.g. `m20260521_add_flow_log_tables`). On each startup, SeaORM compares the migration list in code against the table — migrations not yet in the table are run in order. Migrations already recorded are skipped. **Existing data is never touched.**

This means the same startup path handles both cases without any conditional logic:

Scenario	What happens
Fresh install, DB just created	`seaql_migrations` empty → all migrations run → flow tables created
Dev machine, up to date	Migration already in `seaql_migrations` → skipped
Production, real data, older release	Migration NOT in `seaql_migrations` → runs once safely → flow tables added
Production, re-deployed same version	Migration already in `seaql_migrations` → skipped

0c. Developer action required to ship the tables

Two files must be edited and one new file must be created:

1. **`api/migration/sql/webview_t3_device_schema.sql`** — add the DDL at the end. This file is embedded into the binary and used only when creating a database from scratch. Using `CREATE TABLE IF NOT EXISTS` here means it is idempotent — even if the migration also runs, there is no conflict.

2. **`api/migration/src/m20260521_add_flow_log_tables.rs`** — new migration file. This is what runs on all existing production databases. Pattern is identical to `m20260403_add_raw_calibration_fields.rs`: raw SQL via `db.execute_unprepared()`, `CREATE TABLE IF NOT EXISTS`, `down()` left as a no-op (SQLite `DROP TABLE` would lose data with no benefit).

3. **`api/migration/src/lib.rs`** — register the new migration inside `T3DeviceMigrator`.

That is all. No changes to `utils.rs`, `db_connection.rs`, or any startup code — the existing `run_t3_device_migrations()` call automatically picks it up.

0d. New migration file skeleton (for implementation)

RUST

```rust
// api/migration/src/m20260521_add_flow_log_tables.rs
use sea_orm_migration::{async_trait::async_trait, prelude::*};

#[derive(DeriveMigrationName)]
pub struct Migration;

#[async_trait]
impl MigrationTrait for Migration {
    async fn up(&self, manager: &SchemaManager) -> Result<(), DbErr> {
        let db = manager.get_connection();

        // T3_FLOW: one row per flow instance
        db.execute_unprepared(
            "CREATE TABLE IF NOT EXISTS T3_FLOW (
                id           INTEGER PRIMARY KEY AUTOINCREMENT,
                flow_id      TEXT    NOT NULL UNIQUE,
                flow_type    TEXT    NOT NULL,
                trigger_src  TEXT    NOT NULL,
                started_at   INTEGER NOT NULL,
                ended_at     INTEGER,
                status       TEXT    NOT NULL DEFAULT 'running',
                hostname     TEXT,
                total_steps  INTEGER NOT NULL DEFAULT 0,
                done_steps   INTEGER NOT NULL DEFAULT 0,
                error_count  INTEGER NOT NULL DEFAULT 0,
                meta         TEXT
            )"
        ).await?;

        db.execute_unprepared(
            "CREATE INDEX IF NOT EXISTS idx_t3_flow_type    ON T3_FLOW (flow_type)"
        ).await?;
        db.execute_unprepared(
            "CREATE INDEX IF NOT EXISTS idx_t3_flow_started ON T3_FLOW (started_at DESC)"
        ).await?;
        db.execute_unprepared(
            "CREATE INDEX IF NOT EXISTS idx_t3_flow_status  ON T3_FLOW (status)"
        ).await?;
```

```
// T3_FLOW_STEP: one row per step inside a flow
db.execute_unprepared(
    "CREATE TABLE IF NOT EXISTS T3_FLOW_STEP (
        id           INTEGER PRIMARY KEY AUTOINCREMENT,
        flow_id      TEXT    NOT NULL,
        seq          INTEGER NOT NULL,
        step_name    TEXT    NOT NULL,
        level        TEXT    NOT NULL DEFAULT 'info',
        source       TEXT,
        api_path     TEXT,
        action_type  INTEGER,
        status       TEXT    NOT NULL DEFAULT 'ok',
        duration_ms  INTEGER,
        payload_ref  TEXT,
        message      TEXT,
        details      TEXT,
        ts_unix      INTEGER NOT NULL,
        ts_fmt       TEXT    NOT NULL
    )"
).await?;

db.execute_unprepared(
    "CREATE INDEX IF NOT EXISTS idx_t3_flow_step_flow ON T3_FLOW_STEP (flow_id)"
).await?;
db.execute_unprepared(
    "CREATE INDEX IF NOT EXISTS idx_t3_flow_step_ts   ON T3_FLOW_STEP (ts_unix D
).await?;

// T3_FLOW_PAYLOAD: tracks offloaded large-payload files
db.execute_unprepared(
    "CREATE TABLE IF NOT EXISTS T3_FLOW_PAYLOAD (
        id           INTEGER PRIMARY KEY AUTOINCREMENT,
        flow_id      TEXT    NOT NULL,
        step_id      INTEGER NOT NULL,
        file_path    TEXT    NOT NULL,
        size_bytes   INTEGER NOT NULL,
        created_at   INTEGER NOT NULL,
        purged       INTEGER NOT NULL DEFAULT 0
    )"
).await?;
```

```
        Ok(())
    }

    async fn down(&self, _manager: &SchemaManager) -> Result<(), DbErr> {
        // Intentionally empty: dropping these tables would lose diagnostic history.
        // To remove manually: DROP TABLE T3_FLOW_PAYLOAD; DROP TABLE T3_FLOW_STEP; DROP
        Ok(())
    }
}
```

And register it in `lib.rs`:

RUST

```rust
pub struct T3DeviceMigrator;

impl MigratorTrait for T3DeviceMigrator {
    fn migrations() -> Vec<Box<dyn MigrationTrait>> {
        vec![
            Box::new(m20260403_add_raw_calibration_fields::Migration),
            Box::new(m20260521_add_flow_log_tables::Migration),   // ← new
        ]
    }
}
```

---

1. Why the tables are still needed

The `FlowHandle` being passed as a parameter is about **how you write** — not **whether you store**.

The tables are the persistent store. Without them there is nothing to query, page through, or drill into. The FlowHandle is just a lightweight in-memory context (a UUID + a db reference + a step counter) that knows which row to write under. Without the tables, the FlowHandle has nowhere to persist anything.

TEXT

```
FlowHandle (memory)          SQLite tables (persistent)
___________________          ____________________________

flow_id: "a3f1..."           T3_FLOW  row for "a3f1..."
step_counter: 3         ──►  T3_FLOW_STEP rows 1, 2, 3...
db: &DatabaseConnection      T3_FLOW_PAYLOAD rows if large
```

2. Confirmed system architecture (from code inspection)

2a. Two FFI entry points (both must be tracked)

From code inspection there are two separate services that call the C++ FFI function:

Entry point	File	Used by
`call_handle_webview_msg()` in `t3_ffi_sync_service.rs`	background sync scheduler	SYNC_CYCLE, GET_PANELS_LIST
`T3000FfiApiService::call_ffi()` in `t3_ffi_api_service.rs`	HTTP request handler	every user-triggered action from React

Both use the same global `BACNETWEBVIEW_HANDLE_WEBVIEW_MSG_FN` pointer and the same `ffi_call_lock()` mutex. Neither should be modified. Only their callers instrument flow steps.

2b. React call chain for user-triggered refreshes

```
TEXT

User clicks Refresh (Inputs page)
  └── InputRefreshApi.refreshAllFromDevice(serialNumber)
        └── PanelDataRefreshService.refreshFromDevice({ type: "input", ... })
              └── T3Transport.refreshDeviceRecords(serial, entryType)  ← Action 17
                    └── POST /api/ffi  (HTTP to Rust)
                          └── T3000FfiApiService.call_ffi(message)
                                └── call_handle_webview_msg(17, buffer)  ← C++ FFI
```

This is the shared core path. ALL point types (inputs, outputs, variables, programs, schedules, PIDs, alarms, trendlogs, graphics) go through `PanelDataRefreshService` → `T3Transport` → `POST /api/ffi`.

2c. Background sync path

```
TEXT

Tokio timer fires every N seconds
  └── FfiSyncService::run_sync_cycle()
        ├── get_panels_list_via_ffi()          ← Action 4
        └── per device: sync_device(serial)
              ├── get_logging_data()            ← Action 15 (background only)
              └── write inputs/outputs/variables to SQLite/MSSQL
```

2d. Existing T3_APP_LOG schema (confirmed from migration SQL)

```SQL
CREATE TABLE IF NOT EXISTS T3_APP_LOG (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    ts_unix  INTEGER,
    ts_fmt   TEXT,
    level    TEXT,
    category TEXT,
    source   TEXT,
    hostname TEXT,
    device_serial TEXT,
    message  TEXT,
    details  TEXT
);
```

This table stays unchanged. New tables are additive.

---

3. New database tables

All stored in `webview_t3_device.db` (local SQLite) only. Never written to MSSQL.

Table: `T3_FLOW`

One row per flow instance. Created when a flow starts.

SQL

```sql
CREATE TABLE IF NOT EXISTS T3_FLOW (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    flow_id      TEXT    NOT NULL UNIQUE,   -- UUID v4, generated at flow start
    flow_type    TEXT    NOT NULL,          -- see section 4 for valid values
    trigger_src  TEXT    NOT NULL,          -- "startup" | "scheduler" | "user" | "api"
    started_at   INTEGER NOT NULL,          -- unix epoch ms
    ended_at     INTEGER,                   -- null until done() called
    status       TEXT    NOT NULL DEFAULT 'running', -- "running"|"ok"|"partial"|"failed
    hostname     TEXT,
    total_steps  INTEGER NOT NULL DEFAULT 0, -- expected step count set at start
    done_steps   INTEGER NOT NULL DEFAULT 0,
    error_count  INTEGER NOT NULL DEFAULT 0,
    meta         TEXT                       -- optional JSON: e.g. {"device_serial":"12
);

CREATE INDEX IF NOT EXISTS idx_t3_flow_type    ON T3_FLOW (flow_type);
CREATE INDEX IF NOT EXISTS idx_t3_flow_started ON T3_FLOW (started_at DESC);
CREATE INDEX IF NOT EXISTS idx_t3_flow_status  ON T3_FLOW (status);
```

Table: `T3_FLOW_STEP`

One row per step. Multiple rows per flow.

SQL

```sql
CREATE TABLE IF NOT EXISTS T3_FLOW_STEP (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    flow_id      TEXT    NOT NULL,   -- FK → T3_FLOW.flow_id
    seq          INTEGER NOT NULL,   -- step order: 1, 2, 3…
    step_name    TEXT    NOT NULL,   -- "dll_load" | "ffi_call" | "db_write" etc.
    level        TEXT    NOT NULL DEFAULT 'info', -- "info"|"warn"|"error"
    source       TEXT,               -- rust module name or react component
    api_path     TEXT,               -- optional: HTTP path e.g. "/api/ffi"
    action_type  INTEGER,            -- optional: FFI action number e.g. 4, 15, 17
    status       TEXT    NOT NULL DEFAULT 'ok', -- "ok"|"skip"|"fail"
    duration_ms  INTEGER,            -- elapsed time for this step
    payload_ref  TEXT,               -- NULL or relative file path if payload offloaded
    message      TEXT,
    details      TEXT,               -- short summary; NULL if large payload offloaded t
    ts_unix      INTEGER NOT NULL,
    ts_fmt       TEXT    NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_t3_flow_step_flow ON T3_FLOW_STEP (flow_id);
CREATE INDEX IF NOT EXISTS idx_t3_flow_step_ts   ON T3_FLOW_STEP (ts_unix DESC);
```

Table: `T3_FLOW_PAYLOAD`

Tracks offloaded large payload files. Only created when `details` exceeds the threshold.

SQL

```sql
CREATE TABLE IF NOT EXISTS T3_FLOW_PAYLOAD (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    flow_id      TEXT    NOT NULL,
    step_id      INTEGER NOT NULL,   -- FK → T3_FLOW_STEP.id
    file_path    TEXT    NOT NULL,   -- relative: "T3WebLog/payloads/YYYY-MM/flowid_seq.
    size_bytes   INTEGER NOT NULL,
    created_at   INTEGER NOT NULL,
    purged       INTEGER NOT NULL DEFAULT 0  -- 0=exists, 1=file deleted
);
```

---

4. Flow types and their steps

DLL_INIT

Triggered once at service startup. Source: Rust.

seq	step_name	source	what it records
1	dll_search	ffi_init	paths tried for T3000.exe
2	dll_load	ffi_init	LoadLibraryA result
3	fn_bind_bacnet	ffi_init	BacnetWebView_HandleWebViewMsg bound ok/fail
4	fn_bind_settings	ffi_init	GetDeviceBasicSettings bound ok/fail
5	db_connect_sqlite	server	local SQLite opened
6	db_connect_mssql	server	MSSQL pool created or skipped (standalone)
7	services_start	server	HTTP 9103, 9104, WebSocket started
8	schema_migration	server	migration check result
9	init_done	server	total duration, any errors

SYNC_CYCLE

Triggered by background scheduler. Source: Rust. One flow per cycle.

seq	step_name	source	what it records
1	cycle_start	ffi_sync	interval_secs, timestamp
2	writer_selected	ffi_sync	target: center_mssql / local_sqlite
3	rediscover_check	ffi_sync	should GET_PANELS_LIST run? yes/no + reason
4	get_panels_list	ffi_sync	action=4, attempt N/M, result count, duration
5	device_sync_{serial}	ffi_sync	per device: ok/fail, points written, duration
6	trendlog_sync	ffi_sync	records written, duration
7	cycle_done	ffi_sync	N/M devices ok, total duration, error_count

Note: step 5 repeats once per device. `meta` on the flow carries `{"total_devices": N}`.

INPUTS_LOAD (and OUTPUTS_LOAD, VARIABLES_LOAD)

Triggered by user page load or refresh button. Spans React + Rust.

seq	step_name	source	what it records
1	page_open	React/InputsPage	device_serial, trigger: "mount"|"refresh"
2	api_dispatch	React/PanelDataRefreshService	action=17, entryType=1, index range
3	ffi_received	t3_ffi_api	message parsed, action confirmed
4	ffi_call	t3_ffi_api	call_handle_webview_msg(17), duration
5	ffi_response	t3_ffi_api	bytes received (payload offloaded if > 4096)
6	db_write	t3_ffi_api	records saved to SQLite, count, duration
7	ui_render	React/InputsPage	items rendered, duration

MESSAGE_ACTION (generic user-triggered FFI call)

Triggered by any React component sending a message to `/api/ffi`. Used when no dedicated flow type applies.

seq	step_name	source	what it records
1	action_received	t3_ffi_api	action number, device_serial
2	ffi_call	t3_ffi_api	call_handle_webview_msg(N), duration
3	ffi_response	t3_ffi_api	bytes, status (payload offloaded if large)
4	db_write	t3_ffi_api	optional: if response triggers a save

CONFIG_CHANGE

Triggered when user saves any setting.

seq	step_name	source	what it records
1	user_submit	React	form name, fields changed (no values — only keys)
2	api_received	config_api	endpoint, payload size
3	validate	config_api	pass/fail, reason
4	db_write	config_api	APPLICATION_CONFIG rows updated
5	history_write	config_api	APPLICATION_CONFIG_HISTORY row
6	service_notify	server	which runtime service acknowledged

USER_AUTH

Triggered on login/logout.

seq	step_name	source
1	login_attempt	auth
2	credential_check	auth
3	session_create	auth

5. Rust FlowHandle design

New file: `api/src/logging/flow.rs`

Struct

RUST

```rust
pub struct FlowHandle {
    pub flow_id: String,          // UUID
    pub flow_type: String,
    db: DatabaseConnection,
    step_counter: AtomicU32,      // auto-increments; caller does not manage seq manual
    payload_threshold: usize,     // default 4096 bytes
}
```

Public API

RUST

```rust
impl FlowHandle {
    /// Create a new flow. Inserts one T3_FLOW row immediately.
    pub async fn start(
        db: &DatabaseConnection,
        flow_type: &str,
        trigger_src: &str,
        total_steps: u32,
        meta: Option<&str>,        // optional JSON string
    ) -> FlowHandle;

    /// Record one step. Inserts T3_FLOW_STEP row.
    /// If `details` > payload_threshold bytes → offload to file, insert T3_FLOW_PAYLOAD
    pub async fn step(
        &self,
        step_name: &str,
        level: &str,               // "info" | "warn" | "error"
        source: &str,
        status: &str,              // "ok" | "skip" | "fail"
        duration_ms: i64,
        message: &str,
        details: Option<&str>,     // large payloads auto-offloaded to file
        api_path: Option<&str>,
        action_type: Option<i32>,
    );

    /// Mark flow finished. Updates T3_FLOW row: status, ended_at, done_steps, error_cou
    pub async fn done(&self, status: &str);

    /// Attach to an existing flow created by another caller (used for cross-boundary fl
    pub async fn resume(db: &DatabaseConnection, flow_id: &str) -> Option<FlowHandle>;
}
```

Payload offload logic (inside step())

```
TEXT
```

```
if details.len() > payload_threshold:
    dir  = T3000_RUNTIME/T3WebLog/payloads/YYYY-MM/
    file = {flow_id}_{seq}.json
    write file to disk
    insert T3_FLOW_PAYLOAD row
    store file path in T3_FLOW_STEP.payload_ref
    set  T3_FLOW_STEP.details = NULL
else:
    store details inline in T3_FLOW_STEP.details
    no file written
```

---

6. Cross-boundary flows (React + Rust sharing one flow_id)

For flows that span React and Rust (INPUTS_LOAD, CONFIG_CHANGE, MESSAGE_ACTION):

1. React generates a UUID at the moment the user action triggers.
2. React sends it with every HTTP request as a custom header: `X-Flow-Id: <uuid>`.
3. Rust API handler extracts it and calls `FlowHandle::resume(db, flow_id)`.
4. Rust records its steps under the same `flow_id`.
5. React records its own steps by calling `POST /api/flows/{flow_id}/client-step`.

React flow step payload:

```
JSON
```

```
{
  "seq": 1,
  "step_name": "page_open",
  "source": "InputsPage",
  "status": "ok",
  "duration_ms": 0,
  "message": "User opened Inputs page",
  "details": null
}
```

For backend-only flows (DLL_INIT, SYNC_CYCLE) the `X-Flow-Id` header is never involved. Rust owns the full flow from start to done.

---

7. New API endpoints

New router: `api/src/logging/flow_api.rs`

TEXT

```
GET  /api/flows
     ?type=DLL_INIT&status=failed&from=1716000000&to=1716086400&limit=50&page=0

GET  /api/flows/:flow_id
     → returns T3_FLOW row + all T3_FLOW_STEP rows for that flow

GET  /api/flows/:flow_id/payload/:step_id
     → streams the payload file if it exists and is not purged

POST /api/flows/:flow_id/client-step
     → React records a client-side step under an existing flow

GET  /api/flows/types
     → returns distinct flow_type values with counts

POST /api/flows/purge
     body: { "older_than_days": 30 }
     → deletes T3_FLOW + T3_FLOW_STEP rows and payload files older than N days
```

All read from local SQLite. No MSSQL interaction.

---

8. Payload cleanup

Cleanup runs on two triggers:

- Scheduled: once every 24 hours inside the existing background task loop.
- Manual: user clicks Purge on the Flow Log page.

Config keys in `APPLICATION_CONFIG`:

- `flow_log.retention_days` — default 30
- `flow_log.payload_size_threshold` — default 4096 (bytes)
- `flow_log.enabled` — default true (disable per flow_type: `flow_log.enabled.DLL_INIT` etc.)

Cleanup steps:

1. Query `T3_FLOW_PAYLOAD WHERE created_at < cutoff AND purged = 0`.

2. For each row: delete file from disk, set `purged = 1`.
3. Delete `T3_FLOW_STEP` and `T3_FLOW` rows older than cutoff (cascade).
4. Keep `T3_FLOW_PAYLOAD` rows with `purged = 1` for N more days as audit trail, then delete.

The `T3_FLOW` table is capped at 10,000 rows via a trim trigger (same pattern as T3_APP_LOG's 5,000 row cap) to prevent unbounded growth even if scheduled cleanup fails.

---

9. New UI page

Location and navigation

`src/t3-react/features/logs/pages/FlowLogPage.tsx`

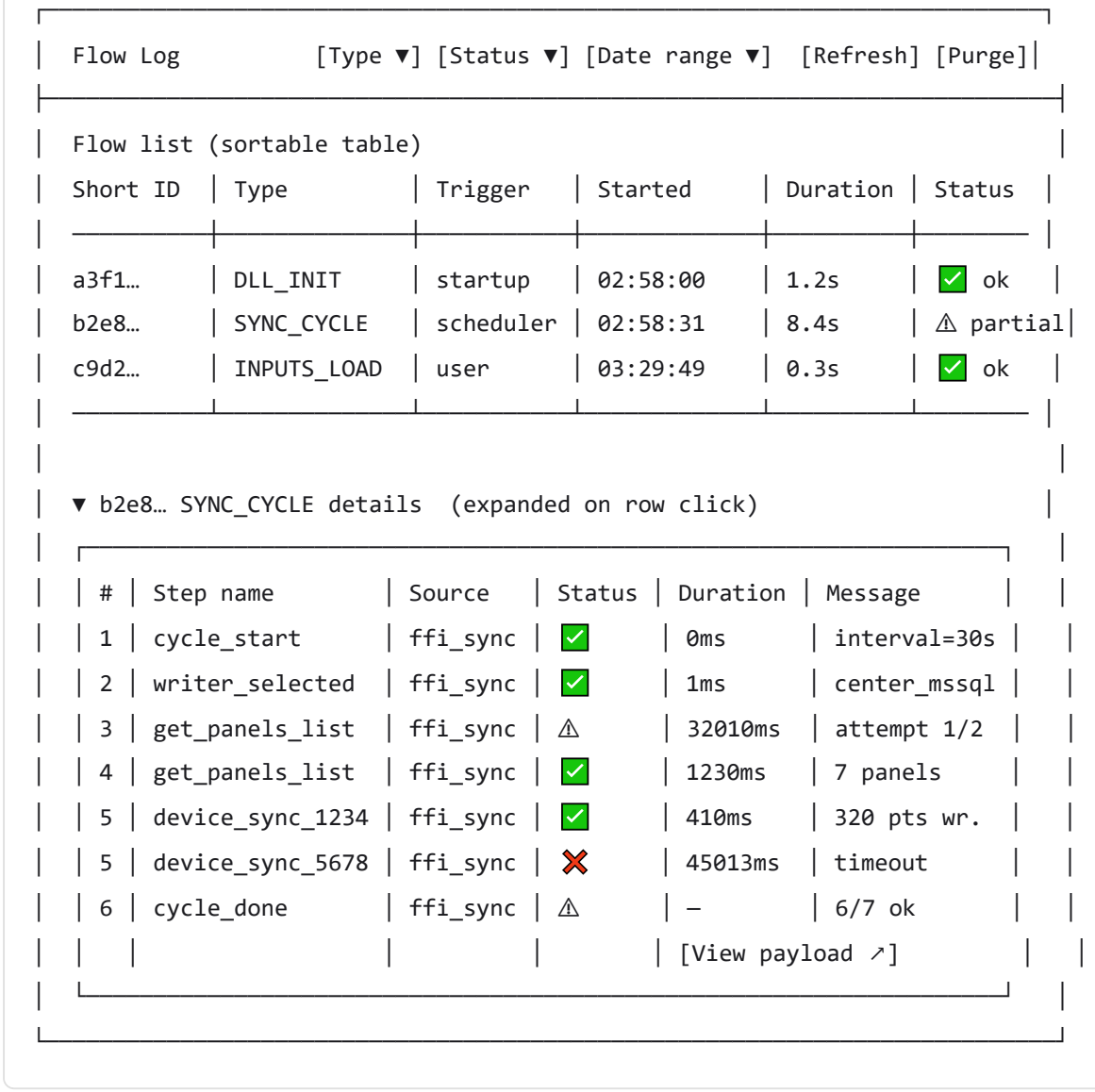
Added as a new tab in the existing Logs page header:

```
TEXT

[ Activity Log ]  [ Flow Log ]  [ File Logs ]  [ Settings ]
```

Layout

TEXT



Interactions

- Click any flow row → expand inline step table
- Click "View payload" on a step → open JSON viewer drawer (streams from `/api/flows/{id}/payload/{step_id}`)
- Purge button → modal with retention config, then calls `POST /api/flows/purge`
- Export button → download full flow as JSON (client-side serialise from already-loaded data)
- Type/Status/Date filters are applied via query params to the API

---

10. What is NOT changed

The following are explicitly left untouched by this design:

Item	Reason
`T3_APP_LOG` table	Still used by existing ActivityLog/sync health system
All `emit_app_log()` call sites	No changes to existing log emission
`ActivityLogTab` component	Still reads from `/api/sync/event-log`
`FileLogsTab`, `LogSettingsTab`	No changes
MSSQL mirror logic in `sinks.rs`	Unchanged for existing categories
`call_handle_webview_msg()` core FFI	Never modified — callers log, not the core
`T3000FfiApiService::call_ffi()`	Never modified — callers log, not the service

11. Implementation order

1. **Migration**: Add `T3_FLOW`, `T3_FLOW_STEP`, `T3_FLOW_PAYLOAD` to `webview_t3_device_schema.sql` + migration runner.
2. **FlowHandle**: `api/src/logging/flow.rs` — start / step / done / resume.
3. **Flow API router**: `api/src/logging/flow_api.rs` — GET flows, GET flow/{id}, POST client-step, POST purge.
4. **Wire into server**: add flow router to `create_t3_app()` in `server.rs`.
5. **Instrument DLL_INIT**: wrap startup in `t3_ffi_api_service.rs` and `main.rs`.
6. **Instrument SYNC_CYCLE**: wrap `run_sync_cycle()` in `t3_ffi_sync_service.rs`.
7. **X-Flow-Id middleware**: extract header in API handler, pass to FlowHandle::resume.
8. **Instrument INPUTS_LOAD** (and OUTPUTS/VARIABLES): wrap `t3_ffi_api_service::call_ffi` caller.
9. **React client-step hook**: small `useFlowLog(flowId)` hook that sends steps to `/api/flows/{id}/client-step`.
10. **FlowLogPage.tsx**: new React page with flow list + step drill-down.
11. **Cleanup scheduler**: wire 24h purge into existing background task in `server.rs`.
12. **Instrument remaining flows**: CONFIG_CHANGE, USER_AUTH, MESSAGE_ACTION.